06 · Quality Management Plan: writing guide

Owner suggestion: Member 3. **Length:** 2 to 3 pages. **Time:** 30 to 40 minutes.

Purpose

Explain **how you planned for and controlled quality**: what "good enough" means for this product, how you checked it, how defects were handled, and what the results were. The brief lists "Quality Management Plan" as a final deliverable.

Key ideas explained simply

- **Quality:** the product does what users need (fitness for use) *and* meets the requirements and standards you set.
- **Quality planning:** deciding the quality objectives, standards and how to measure them, *before* building.
- **Quality assurance (QA):** checking the **process** is good (reviews, retrospectives, coding standards, using Git, CI).
- **Quality control (QC):** checking the **product** is good (testing, inspections, bug tracking).
- **Defect:** something that does not meet a requirement. Track each one: found, logged, prioritised, fixed, retested, closed.
- **Acceptance criteria:** the conditions a user story must meet to count as done.
- **Definition of Done (DoD):** the team's checklist for "finished" (code written, reviewed, tested, documented).
- **Metric:** a number you can track, so quality is not just opinion.

Sections to write (in this order)

1. Quality objectives (a short table)

What does quality mean for CPUTrade? Examples to adapt: *trustworthy* (verified identity and ratings), *secure* (data and accounts protected), *usable* (mobile-first, simple flows), *reliable* (no data corruption, works in the demo), *maintainable* (organised modules, documented, tested).

Quality objective	Metric / target	How measured	How often	Owner
Example: Security rules work as specified	All automated security tests pass	Run `./mvnw test`	Every change	

Metrics you can honestly report: automated tests passing (25 of 25); manual scenarios passed (your 15-case table); defects found and fixed (at least one major: the concurrency bug); user stories completed versus planned (from your sprint records); number of code reviews or retrospectives held (your records). Only use numbers you can show.

2. Standards and conventions (a short list)

Examples that exist in the project: REST-style endpoints; code organised into feature modules (user, product, chat, trust, rating, notification, bulletin, admin); input validation on every request (email domain, password length, image type and size); passwords hashed; consistent error messages; mobile-first responsive design (works down to 320 px); accessibility touches (focus rings, reduced-motion support); Git for all code with meaningful commit messages. Add what your team really agreed (naming, branch rules, review rules).

3. Quality assurance activities (process)

List the activities, who did them and when: sprint planning with acceptance criteria; peer review of code or pull requests; sprint reviews and retrospectives; continuous integration (a GitHub workflow builds and tests on every push; say plainly if you have not yet seen it run); documentation kept in the repository (README, DEPLOYMENT.md).

4. Quality control activities (product)

- **Automated tests:** 25 backend tests covering registration rules (including lookalike email domains), vendor approval, password hashing, login errors, token validity/expiry/forgery, image validation, login rate limiting, production secrets check.
- **Manual scenario testing:** the 15 test cases (see the Final Report guide) covering registration, listing, chat, the verified sale, ratings, moderation, suspension, wishlist alerts and installing the app.
- **Security checks done live:** brute-force login lockout, fake and oversized uploads, concurrency test.
- **Usability checks:** phone-size testing, demo walk-through with the team (add any feedback you collected from real users).
- **Not done (be honest):** independent penetration test; automated tests for chat, sales and admin flows; load/performance testing.

5. Defect management

Describe the process in a few steps: how a defect is found, logged (where: issue list, notes, git), prioritised (severity), fixed, retested and closed. Then include a short defect log:

ID	Defect	Severity	Found by / how	Fix	Retest result
D1	*Example:* Simultaneous "mark sold" requests reduced stock by one instead of six	High (data accuracy)	Concurrency test	Database row lock	Stock went 50 to 44 as expected
D2					

Another real example: the first lock approach used a syntax the MariaDB database rejected; replaced by a standard `SELECT ... FOR UPDATE` and retested. Add defects from your own sprint records.

6. Roles and responsibilities

Who owned quality? (For example one member as QA lead, all members as developers who test their own work.) Match your RACI.

7. Acceptance and sign-off

When is the product accepted? Suggest: all "Must" requirements done, all automated tests passing, manual scenarios passed or defects logged with a plan, demo succeeds, documentation complete. Say who signs off (lecturer for assessment, team for internal acceptance).

8. Continuous improvement

Two or three lines on how retrospectives and defects changed your way of working (link to your Reflection). Then a short "Quality next steps" list: add more automated tests (chat, sales, admin), independent penetration test, performance testing, accessibility audit.

Common mistakes

- Describing quality only as "testing". Include process (QA) as well as product (QC).
- Targets with no measurement ("high quality").
- Claiming tests or reviews you did not do.
- No defect log or no evidence of fixes.

Checklist

- [ ] Objectives with metrics and how measured
- [ ] Standards and conventions
- [ ] QA (process) and QC (product) activities
- [ ] Defect process and at least one real defect with its fix
- [ ] Roles, acceptance criteria and sign-off
- [ ] Honest list of what was not tested